

LAPORAN TUGAS BESAR 2
IF3070 Dasar Inteligensi Artifisial
Implementasi Algoritma Pembelajaran Mesin

Dosen Pengampu: Ir. Windy Gambetta, M.B.A.

Kelas / Kelompok : K-01 / 16



Disusun oleh:

Carlen Asadel Axelle	18223017
Devon Wiraditya Tanumihardja	18223039
Ananda Fajrul Zidan	18223067
Adam Joaquin Girsang	18223089

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

DAFTAR ISI

DAFTAR ISI.....	2
BAB I	
Pendahuluan.....	3
A. Latar Belakang.....	3
B. Tujuan.....	3
BAB II	
Deskripsi Implementasi.....	5
A. Classification and Regression Tree (CART).....	5
1. Dasar Teori.....	5
2. Implementasi Algoritma tanpa library.....	5
3. Langkah Implementasi.....	6
B. Logistic Regression.....	7
1. Dasar Teori.....	7
2. Implementasi Algoritma tanpa library.....	7
3. Langkah Implementasi.....	8
BAB III.....	10
Data Cleaning.....	10
A. Data Cleaning.....	10
1. Missing Values.....	10
2. Outliers.....	13
B. Data Preprocessing.....	13
1. Feature Scaling.....	13
2. Feature Encoding.....	14
BAB IV.....	17
Feature Engineering.....	17
A. Feature Selection & Binning (Discretization).....	17
1. Feature Selection.....	17
2. Binning (Discretization).....	17
BAB V	
Kesimpulan dan Saran.....	21
A. Kesimpulan.....	21
B. Saran.....	22
Pembagian Tugas.....	24
REFERENSI.....	25

BAB I

Pendahuluan

A. Latar Belakang

Pembelajaran mesin (*Machine Learning*) merupakan salah satu cabang dari kecerdasan buatan yang memungkinkan sistem untuk belajar dari data dan membuat prediksi atau keputusan tanpa diprogram secara eksplisit. Dalam penerapannya, algoritma pembelajaran mesin sangat berguna untuk menangani masalah dunia nyata yang kompleks, salah satunya adalah deteksi kecurangan (*fraud detection*).

Pada Tugas Besar 2 mata kuliah IF3070 Dasar Inteligensi Artifisial ini, fokus utama diberikan pada implementasi algoritma pembelajaran mesin untuk mengklasifikasikan data. Dataset yang digunakan adalah data sintetis berskala besar (sekitar 100.000 baris) yang dirancang menyerupai pola *fraud* nyata, yang mencakup tantangan seperti adanya *outlier*, *missing values*, serta interaksi antar-fitur yang kompleks.

Tantangan teknis dalam tugas ini tidak hanya terletak pada penggunaan pustaka (*library*) yang sudah ada, tetapi juga pada pemahaman mendalam mengenai logika algoritma melalui implementasi *from scratch*. Penulis diminta untuk membangun model *Decision Tree Learning* (DTL) serta memilih antara *Logistic Regression* atau *K-Nearest Neighbors* (KNN). Selain itu, perbandingan performa antara implementasi manual dan implementasi menggunakan pustaka *scikit-learn* menjadi krusial untuk memvalidasi kebenaran logika dan efisiensi algoritma yang dibangun.

B. Tujuan

Tujuan dari tugas ini adalah:

1. Membangun pustaka mini (*library*) pembelajaran mesin yang berisi implementasi *Decision Tree* dan *Logistic Regression*.

2. Menghasilkan model klasifikasi *fraud* yang akurat dan dapat disimpan (*saved models*) untuk penggunaan masa depan.
3. Memahami secara mendalam logika matematis di balik algoritma klasifikasi melalui implementasi manual (*from scratch*).
4. Mengevaluasi kinerja model menggunakan metrik yang relevan (akurasi, F1-score) dan membandingkannya dengan standar industri.

BAB II

Deskripsi Implementasi

A. Classification and Regression Tree (CART)

1. Dasar Teori

Classification and Regression Tree (CART) adalah algoritma decision tree yang menghasilkan pohon biner (setiap node hanya memiliki dua cabang). Berbeda dengan ID3 yang menggunakan Entropy dan Information Gain, CART menggunakan Gini Impurity sebagai kriteria pemisahan (splitting criterion).

Tujuan utama algoritma adalah meminimalkan impurity di setiap percabangan. Nilai Gini Impurity untuk sebuah node dihitung dengan rumus:

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

2. Implementasi Algoritma tanpa *library*

Fungsi	Deskripsi
<code>def __init__(self, learning_rate=0.01, n_iterations=1000, ...)</code>	Menginisialisasi model Logistic Regression dengan parameter pembelajaran (learning rate), jumlah iterasi, dan jenis regularisasi (L1/L2) untuk mencegah overfitting.

<code>def _sigmoid(self, z)</code>	Menghitung nilai fungsi aktivasi Sigmoid. Fungsi ini mengubah output linear menjadi probabilitas antara 0 dan 1.
<code>def _compute_cost(self, X, y)</code>	Menghitung total error (biaya) model saat ini menggunakan rumus Log-Loss (Binary Cross-Entropy), ditambah dengan penalti regularisasi jika diaktifkan.
<code>def fit(self, X, y)</code>	Melatih model menggunakan metode Gradient Descent. Melakukan iterasi sebanyak <code>n_iterations</code> untuk memperbarui bobot dan bias agar fungsi biaya menjadi minimal.
<code>def predict_proba(self, X)</code>	Menghitung probabilitas bahwa data input X termasuk dalam kelas positif (1) menggunakan bobot yang telah dilatih.
<code>def predict(self, X, threshold=0.5)</code>	Memprediksi label kelas biner (0 atau 1) untuk data X. Menggunakan hasil dari <code>predict_proba</code> dan membandingkannya dengan nilai ambang batas (<code>threshold</code>).

3. Langkah Implementasi

- a) **Inisialisasi Model:** Membuat objek pohon dengan parameter batasan kedalaman dan jumlah sampel minimum.
- b) **Preprocessing Internal:** Saat `fit()` dipanggil, data dikonversi ke format numpy. Nilai NaN (jika ada) diimputasi dengan nilai median kolom terkait.
- c) **Pembentukan Pohon (Training):**
 - Hitung Gini Impurity saat ini.
 - Iterasi semua fitur dan nilai unik untuk mencari *best split*.
 - Jika *gain* positif dan kriteria kedalaman belum terlampaui, data dipecah menjadi *left* dan *right*.
 - Lakukan langkah yang sama secara rekursif untuk cabang kiri dan kanan.

- d) **Prediksi:** Data input ditelusuri melewati node-node keputusan berdasarkan nilai fiturnya hingga sampai di node daun (*leaf*) yang memuat nilai prediksi kelas mayoritas.

B. Logistic Regression

1. Dasar Teori

Logistic Regression adalah algoritma klasifikasi yang memprediksi probabilitas kejadian suatu kelas biner (0 atau 1). Algoritma ini menggunakan fungsi linear $z = w \cdot x + b$ yang kemudian dipetakan ke rentang 0 hingga 1 menggunakan Fungsi Sigmoid:

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

Algoritma meminimalkan fungsi biaya (Cost Function) yang disebut Log-Loss atau Binary Cross-Entropy menggunakan metode optimasi Gradient Descent.

2. Implementasi Algoritma tanpa *library*

Fungsi	Deskripsi
--------	-----------

<code>def __init__(self, learning_rate=0.01, n_iterations=1000, ...)</code>	Menginisialisasi parameter pembelajaran seperti <code>learning_rate</code> , <code>n_iterations</code> , serta parameter regularisasi (<code>lambda_</code> , <code>regularization</code>) untuk mengontrol kompleksitas model.
<code>def fit(self, X, y)</code>	Melatih model menggunakan metode Gradient Descent. Pada setiap iterasi, bobot (w) dan bias (b) diperbarui berlawanan arah dengan gradien error untuk meminimalkan cost.
<code>def _sigmoid(self, z)</code>	Fungsi aktivasi yang mengubah nilai linear menjadi probabilitas antara 0 dan 1. Dilengkapi dengan <code>np.clip</code> untuk menjaga stabilitas numerik (overflow protection).
<code>def _compute_cost(self, X, y)</code>	Menghitung nilai error (Loss) menggunakan rumus Log-Loss. Fungsi ini juga menambahkan nilai penalti jika regularisasi L1 atau L2 diaktifkan.
<code>def predict_proba(self, X)</code>	Menghitung probabilitas data masuk ke kelas positif (1) menggunakan bobot yang telah dilatih dan fungsi sigmoid.
<code>def predict(self, X, threshold=0.5)</code>	Mengonversi hasil probabilitas menjadi label kelas biner (0 atau 1) berdasarkan threshold tertentu (default 0.5).

3. Langkah Implementasi

- Inisialisasi Bobot:** Bobot (w) diinisialisasi dengan nol dan bias (b) dengan 0.
- Forward Propagation:** Menghitung prediksi linear (z) dan mengaktifkannya dengan Sigmoid untuk mendapatkan prediksi probabilitas.
- Cost Calculation:** Menghitung seberapa jauh prediksi dari label asli menggunakan *Log-Loss*, ditambah *penalty* regularisasi jika ada.
- Backward Propagation:** Menghitung gradien (turunan) terhadap bobot dan bias.
- Weight Update:** Memperbarui parameter bobot dan bias menggunakan rumus: $w = w - (\text{learning_rate} * dw)$.

- f) **Iterasi:** Ulangi langkah 2-5 sebanyak jumlah iterasi ($n_{\text{iterations}}$) yang ditentukan.

BAB III

Data Cleaning

A. Data Cleaning

1. Missing Values

Pengisian *missing values* dilakukan menggunakan *domain specific strategies* dari jurnal yang disediakan serta informasi *phising* dan *non-phising* lainnya. Kami menggunakan *pipeline* untuk memudahkan transformasi.

Fungsi	Deskripsi
<pre>columns = ["transaction_amount"] for column in columns: column_data = new_train[column] q1 = np.percentile(column_data, 25) q3 = np.percentile(column_data, 75) IQR = q3 - q1 lower = q1 - 1.5 * IQR upper = q3 + 1.5 * IQR outliers = column_data[(column_data < lower) (column_data > upper)] median = column_data.median() column_data[(column_data < lower) (column_data > upper)] = median</pre>	Median dipilih karena lebih robust terhadap nilai ekstrem dibandingkan mean, sehingga tidak terpengaruh oleh outlier yang sangat besar atau kecil. Perbedaan signifikan antara mean dan median pada transaction_amount mengindikasikan distribusi yang sangat skewed dengan adanya nilai-nilai ekstrem yang menarik mean menjauhi pusat distribusi.
<pre>columns = ["avg_transaction_amount"]</pre>	Mean dipilih sebagai nilai imputasi karena

<pre> for column in columns: column_data = new_train[column] q1 = np.percentile(column_data, 25) q3 = np.percentile(column_data, 75) IQR = q3 - q1 lower = q1 - 1.5 * IQR upper = q3 + 1.5 * IQR outliers = column_data[(column_data < lower) (column_data > upper)] mean = column_data.mean() column_data[(column_data < lower) (column_data > upper)] = mean </pre>	jumlah outlier pada fitur ini relatif sedikit dan merepresentasikan lonjakan aktivitas yang masih dalam konteks normal, bukan anomali ekstrem seperti pada transaction_amount.
<pre> columns = ["device_trust_score"] for column in columns: column_data = new_train[column] q1 = np.percentile(column_data, 25) q3 = np.percentile(column_data, 75) IQR = q3 - q1 lower = q1 - 1.5 * IQR upper = q3 + 1.5 * IQR outliers = column_data[(column_data < lower) (column_data > upper)] mean = column_data.mean() column_data[(column_data < lower) </pre>	Mean dipilih untuk device_trust_score karena skor kepercayaan perangkat biasanya memiliki distribusi yang lebih terbatas dan simetris dalam rentang tertentu, sehingga outlier tidak terlalu ekstrem.

<code>(column_data > upper)] = mean</code>	
---	--

2. Outliers

Fungsi	Deskripsi
<pre>trans_median = new_train['transaction_amount'].median() new_train['transaction_amount'] = new_train['transaction_amount'].fillna(trans_ median)</pre>	Median dipilih karena lebih robust terhadap outlier dan distribusi skewed dibandingkan mean, sehingga nilai imputasi tidak terpengaruh oleh transaksi dengan jumlah ekstrem yang mungkin masih ada dalam data.
<pre>avg_trans_mean = new_train['avg_transaction_amount'].mean() new_train['avg_transaction_amount'] = new_train['avg_transaction_amount'].fillna(av g_trans_mean)</pre>	Mean dipilih untuk imputasi karena outlier pada fitur ini sudah ditangani sebelumnya, sehingga mean sudah representatif dan tidak lagi terdistorsi oleh nilai ekstrem.
<pre>device_trust_mean = new_train['device_trust_score'].mean() new_train['device_trust_score'] = new_train['device_trust_score'].fillna(device_t rust_mean)</pre>	Mean dipilih untuk imputasi karena outlier pada fitur ini sudah ditangani sebelumnya, sehingga mean sudah representatif dan tidak lagi terdistorsi oleh nilai ekstrem.

B. Data Preprocessing

1. Feature Scaling

Fungsi	Deskripsi
<pre>from sklearn.preprocessing import StandardScaler</pre>	Standardisasi diperlukan untuk algoritma yang sensitif terhadap skala fitur, namun

<pre> X_train = X_train.replace([np.inf, -np.inf], np.nan) X_test = X_test.replace([np.inf, -np.inf], np.nan) binary_cols = ["has_chargedback_history", "is_new_country", "transaction_id_fraud_rate"] scale_cols = [col for col in X_train.columns if col not in binary_cols] scaler = StandardScaler() X_train_scaled = X_train.copy() X_test_scaled = X_test.copy() X_train_scaled[scale_cols] = scaler.fit_transform(X_train[scale_cols]) X_test_scaled[scale_cols] = scaler.transform(X_test[scale_cols]) </pre>	kolom binary dan siklik dikecualikan karena sudah dalam skala 0-1 dan scaling akan merusak interpretasi nilai boolean/categorical mereka.
--	--

2. Feature Encoding

Fungsi	Deskripsi
<pre> gender_mapping = {'M': 1, 'F': 0} transformed_new_train['gender'] = transformed_new_train['gender'].map(gender_ mapping) </pre>	Encoding gender dengan mapping {'M': 1, 'F': 0} didasarkan pada analisis distribusi fraud berdasarkan gender, di mana nilai numerik

	yang lebih tinggi (1) diberikan kepada kategori dengan tingkat fraud yang lebih tinggi (Male), sementara nilai lebih rendah (0) untuk kategori dengan risiko fraud lebih kecil (Female).
<pre>country_mapping = {"US": 9, "IN": 8, "ID": 7, "UK": 6, "CA": 5, "BR": 4, "FR": 3, "DE": 2, "SG": 1, "AU": 0} transformed_new_train['country'] = transformed_new_train['country'].map(country_mapping) transformed_new_train</pre>	Encoding country menggunakan mapping ordinal dari 0-9 didasarkan pada analisis tingkat fraud per negara, di mana nilai numerik yang lebih tinggi diberikan kepada negara dengan frekuensi atau rasio fraud yang lebih tinggi, dan nilai lebih rendah untuk negara dengan risiko fraud minimal.
<pre>device_type_mapping = {"mobile": 2, "desktop": 1, "tablet": 0} transformed_new_train['device_type'] = transformed_new_train['device_type'].map(device_type_mapping) transformed_new_train</pre>	Encoding device type menggunakan mapping ordinal {"mobile": 2, "desktop": 1, "tablet": 0} didasarkan pada analisis frekuensi dan pola fraud berdasarkan jenis perangkat, di mana nilai numerik yang lebih tinggi mencerminkan tingkat risiko fraud yang lebih tinggi.
<pre>device_os_mapping = {"Android": 4, "iOS": 3, "Windows": 2, "Linux": 1, "MacOS": 0} transformed_new_train['device_os'] = transformed_new_train['device_os'].map(device_os_mapping) transformed_new_train</pre>	Encoding os type menggunakan mapping ordinal {"Android": 4, "iOS": 3, "Windows": 2, "Linux": 1, "MacOS": 0} didasarkan pada analisis frekuensi dan pola fraud berdasarkan jenis os, di mana nilai numerik yang lebih tinggi mencerminkan tingkat risiko fraud yang lebih tinggi.
<pre>merchant_category_mapping = {"electronics": 8, "restaurants": 7, "gas": 6, "groceries": 5, "travel": 4, "clothing": 3, "financial": 2, "services": 1, "luxury": 0}</pre>	Encoding merchant_category menggunakan mapping ordinal {"electronics": 8, "restaurants": 7, "gas": 6, "groceries": 5,

<pre>transfromed_new_train['merchant_category'] = transfromed_new_train['merchant_category']. map(merchant_category_mapping) transfromed_new_train</pre>	"travel": 4, "clothing": 3, "financial": 2, "services": 1, "luxury": 0} didasarkan pada analisis frekuensi dan pola fraud berdasarkan jenis merchant, di mana nilai numerik yang lebih tinggi mencerminkan tingkat risiko fraud yang lebih tinggi.
<pre>transaction_type_mapping = {"purchase": 3, "withdrawal": 2, "topup": 1, "transfer": 0} transfromed_new_train['transaction_type'] = transfromed_new_train['transaction_type'].ma p(transaction_type_mapping) transfromed_new_train</pre>	Encoding transaction_type menggunakan mapping ordinal {"purchase": 3, "withdrawal": 2, "topup": 1, "transfer": 0} didasarkan pada analisis frekuensi dan pola fraud berdasarkan jenis transaksi, di mana nilai numerik yang lebih tinggi mencerminkan tingkat risiko fraud yang lebih tinggi.

BAB IV

Feature Engineering

A. Feature Selection & Binning (Discretization)

1. Feature Selection

Feature selection (seleksi fitur) merupakan proses identifikasi dan pemilihan subset variabel yang paling signifikan dari sebuah dataset untuk digunakan dalam pengembangan model machine learning. Langkah ini esensial untuk membuang atribut yang redundan, tidak relevan, atau tidak memberikan nilai tambah pada prediksi. Penerapan feature selection memiliki beberapa tujuan utama, yaitu:

- Mereduksi dimensi data agar lebih ringkas dan efisien.
- Mencegah terjadinya overfitting dengan membatasi kompleksitas data.
- Mempercepat waktu komputasi selama proses pelatihan model.
- Menyederhanakan implementasi dan interpretasi model yang dihasilkan.
- Memastikan model hanya berfokus pada fitur-fitur yang memiliki kontribusi relevan.

2. Binning (Discretization)

Binning, atau dikenal juga sebagai diskretisasi, adalah teknik transformasi data yang mengubah variabel numerik kontinu menjadi sejumlah interval atau kategori diskret. Metode ini bertujuan untuk menyederhanakan representasi data numerik dan mengurangi sensitivitas model terhadap fluktuasi data yang ekstrem (outlier). Adapun tujuan spesifik dari teknik binning meliputi:

- Mengurangi dimensi dan variabilitas data yang tidak perlu.

- Menangani outlier secara efektif dengan mengelompokkannya ke dalam rentang kategori tertentu.
- Mempermudah analisis pola data.
- Meningkatkan stabilitas dan kinerja pada algoritma model tertentu.

Fungsi	Deskripsi
<pre>#feature engineering for col in train_cat: fraud_rate = train.groupby(col)["is_fraud"].mean() all_data[col + "_fraud_rate"] = all_data[col].map(fraud_rate)</pre>	Kode tersebut melakukan target encoding untuk fitur kategorikal dengan menghitung rata-rata fraud rate (is_fraud) per kategori di training data, kemudian memetakan nilai tersebut sebagai fitur numerik baru ke seluruh dataset. Walaupun itu, fitur2 ini hanya digunakan untuk menghitung normalisasi dan tidak digunakan dalam training.
<pre>all_data["amount_vs_avg"] = (all_data["transaction_amount"] / (all_data["avg_transaction_amount"] + 1))</pre>	Fitur ini menghitung rasio jumlah transaksi saat ini terhadap rata-rata transaksi historis pengguna, dengan menambahkan 1 pada denominator untuk menghindari division by zero.
<pre>all_data["amount_zscore"] = ((all_data["transaction_amount"] - all_data["avg_transaction_amount"]) / (all_data["std_transaction_amount"] + 1))</pre>	Menghitung z-score statistik yang mengukur seberapa jauh transaksi saat ini menyimpang dari rata-rata dalam satuan standar deviasi, dengan rumus (nilai - mean) / std. Z-score tinggi (>3) atau rendah (<-3) menunjukkan outlier statistik

<pre>all_data["tx_per_hour"] = all_data["transactions_last_1h"] all_data["tx_per_day"] = all_data["transactions_last_24h"] all_data["burst_activity"] = (all_data["transactions_last_1h"] / (all_data["transactions_last_24h"] + 1))</pre>	Menghitung rasio transaksi 1 jam terakhir terhadap 24 jam terakhir, mengidentifikasi konsentrasi aktivitas dalam waktu singkat. Nilai mendekati 1 berarti hampir semua transaksi hari ini terjadi dalam 1 jam terakhir.
<pre>all_data["ip_device_risk_ratio"] = (all_data["ip_risk_score"] / (all_data["device_trust_score"] + 1e-3))</pre>	Membandingkan IP risk score dengan device trust score, dengan menambahkan epsilon (1e-3) untuk stabilitas numerik.
<pre>all_data["merchant_country_risk"] = (all_data["merchant_risk"] * all_data["country_risk"])</pre>	mengalikan merchant risk dan country risk untuk menangkap combined risk effect
<pre>drop_cols = (["ID", "transaction_id", "user_id"] + train_cat.columns.tolist()) train_fe = all_data.drop(columns=drop_cols)</pre>	Mengdrop tabel-tabel yang tidak digunakan dalam training model
<pre># Normalisasai liat ga terdistribusi normal for col in ["transaction_amount", "avg_transaction_amount", "std_transaction_amount", "transaction_duration", "num_prev_transactions", "transactions_last_24h", "transactions_last_1h", "failed_login_attempts", "shared_ip_users", "shared_device_users", "account_age_days", "amount_vs_avg", "amount_zscore", "burst_activity", "ip_device_risk_ratio"]:</pre>	Kode ini menerapkan log transformation menggunakan np.log1p pada 15 fitur numerik yang memiliki distribusi skewed (long-tail) untuk membuat distribusinya lebih mendekati normal. Log transformation diterapkan selektif hanya pada kolom yang memiliki distribusi skewed dengan outlier signifikan

<pre>train_fe[col + "_log"] = np.log1p(train_fe[col])</pre>	
---	--

BAB V

Kesimpulan dan Saran

A. Kesimpulan

Berdasarkan tahap eksplorasi, pra-pemrosesan, hingga implementasi algoritma yang telah dilakukan dalam tugas besar ini, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Implementasi Algoritma *From Scratch* Penulis berhasil**

mengimplementasikan algoritma Classification and Regression Tree (CART) dan Logistic Regression tanpa menggunakan pustaka pembelajaran mesin tingkat tinggi (hanya menggunakan NumPy dan Pandas).

- Pada CART, penggunaan *Gini Impurity* sebagai kriteria pemisahan (*splitting criterion*) terbukti mampu membangun pohon keputusan yang dapat memisahkan data berdasarkan fitur-fitur yang ada.
- Pada Logistic Regression, penerapan *Gradient Descent* dan fungsi *Sigmoid* berhasil melatih model untuk memprediksi probabilitas kelas biner (fraud/non-fraud).

2. **Pentingnya *Data Cleaning* dan *Preprocessing* Kualitas data sangat**

mempengaruhi kinerja model. Tahapan yang telah dilakukan terbukti krusial:

- Penanganan *Missing Values*: Penggunaan strategi imputasi berbasis median untuk fitur yang *skewed* (seperti *transaction_amount*) dan mean untuk fitur yang terdistribusi normal berhasil menjaga integritas data tanpa membuang informasi berharga.
- Penanganan *Outliers*: Metode IQR (*Interquartile Range*) efektif untuk mengidentifikasi dan menangani pencilan yang dapat mendistorsi model, khususnya pada algoritma regresi.

- *Feature Scaling & Encoding*: Standardisasi (StandardScaler) membantu algoritma optimasi bekerja lebih cepat, sementara *encoding* (seperti pada gender, country, device_os) memungkinkan algoritma memproses data kategorikal secara matematis.

3. **Peran *Feature Engineering* Pembuatan fitur baru seperti fraud_rate per kategori dan rasio transaksi (amount_vs_avg)** memberikan dimensi informasi tambahan yang signifikan. Transformasi logaritmik ($\log_{1p}$) pada fitur yang memiliki distribusi *long-tail* juga membantu model regresi dalam menangani data yang tidak terdistribusi normal.

4. **Perbandingan dengan Pustaka (Scikit-Learn) Berdasarkan hasil eksperimen**, model yang dibangun dari awal (*from scratch*) mampu menghasilkan metrik evaluasi yang kompetitif dibandingkan implementasi Scikit-Learn. Namun, dari segi efisiensi waktu, implementasi Scikit-Learn jauh lebih unggul karena optimasi tingkat rendah yang dimilikinya. Hal ini memvalidasi pemahaman logika algoritma penulis sekaligus menunjukkan tantangan efisiensi dalam implementasi manual.

B. Saran

Dalam pengerjaan tugas ini, beberapa aspek implementasi dan metodologi telah berhasil diterapkan dengan baik, namun masih terdapat ruang untuk pengembangan dan peningkatan performa model fraud detection di masa mendatang. Berikut adalah beberapa saran perbaikan dan pengembangan yang dapat dipertimbangkan:

1. Handling Imbalanced Data: Implementasikan teknik SMOTE, ADASYN, atau class weighting untuk mengatasi ketidakseimbangan kelas fraud vs non-fraud, serta prioritaskan metrik F1-score, precision, recall, dan AUC-ROC dibanding accuracy yang bias untuk imbalanced dataset.
2. Feature Selection: Gunakan metode statistik seperti mutual information, chi-square test, atau feature importance dari decision tree untuk mengidentifikasi dan eliminasi fitur redundan atau noise yang tidak berkontribusi pada prediksi.
3. Cross-Validation: Implementasikan stratified k-fold cross-validation ($k=5$ atau 10) untuk evaluasi yang lebih robust dan mengurangi bias dari single train-test split, terutama pada dataset imbalanced.

Pembagian Tugas

Nama Lengkap	NIM	Deskripsi Tugas
Carlen Asadel Axelle	18223017	Membuat Algoritma Logistic Regression
		Mengisi laporan
Devon Wiraditya	18223039	Melakukan Data Training dan Prediction
		Melakukan Data Description dan Exploration
		Melakukan Data Preparation
		Melakukan Data Transformation (Normalization)
		Melakukan Data Scaling
Ananda Fajrul Zidan	18223067	Melakukan Data Description dan Exploration
		Melakukan Data Preparation
		Melakukan Data Transformation (Normalization)
		Melakukan Data Scaling
		Melakukan Data Training dan Prediction
Adam Joaquin Girsang	18223089	<i>Mengisi sebagian besar laporan</i>

REFERENSI

GeeksforGeeks. (2024). *Iterative Dichotomiser 3 (ID3) Algorithm from Scratch*. Diakses dari <https://www.geeksforgeeks.org/machine-learning/iterative-dichotomiser-3-id3-algorithm-from-scratch/>

GeeksforGeeks. (2024). *CART (Classification and Regression Tree) in Machine Learning*. Diakses dari <https://www.geeksforgeeks.org/machine-learning/cart-classification-and-regression-tree-in-machine-learning/>

GeeksforGeeks. (2024). *Decision Tree Algorithms*. Diakses dari <https://www.geeksforgeeks.org/machine-learning/decision-tree-algorithms/>

GeeksforGeeks. (2024). *Support Vector Machine (SVM) Algorithm*. Diakses dari <https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>

GeeksforGeeks. (2024). *Understanding Logistic Regression*. Diakses dari <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>

Jee, K. (n.d.). *Titanic Project Example* [Video]. YouTube. <https://www.youtube.com/watch?v=I3FBJdiExcg>

Jee, K. (n.d.). *Titanic Project Example* [Source code]. Kaggle. <https://www.kaggle.com/code/kenjee/titanic-project-example/notebook>